Guia de inicio de programacion MVC

Por donde empezar, clases sugeridas, acomodos y reglas para PokeRush

Proyecto	Grupo4_PokemonBossRush_ESTFA_2026
Motor	Unity
Lenguaje	C#
Base actual	Estructura MVC vacia en Juego/PokeRush/Assets/Scripts
Proposito	Orientar el comienzo de la programacion sin agregar codigo automaticamente

Nota. Este PDF no implementa funcionalidad. Es una guia para que el equipo decida que crear primero y donde ubicarlo dentro de la estructura MVC.

1. Primer objetivo: un corte vertical minimo

Conviene empezar con una version minima que atraviese todas las capas: datos, modelo, controlador y vista. No hace falta que sea el juego completo; alcanza con una escena que muestre una batalla simple y permita validar el flujo.

Orden	Tarea	Resultado esperado
1	Definir datos basicos	Pokemon del jugador, un boss y valores simples de vida/ataque.
2	Crear modelos minimos	Estado de pokemon y estado de batalla sin UI.
3	Crear controlador de batalla	Coordina inicio, accion del jugador y avance del turno.
4	Crear vista de batalla	Muestra vida, nombre, turno y mensajes.
5	Probar en escena	Confirmar que el flujo funciona antes de sumar mas sistemas.

Consejo: no arrancar por inventario, guardado, menus complejos o muchas animaciones. Primero cerrar un circuito chico que el equipo pueda probar.

2. Mapa de carpetas y dependencias

Carpeta	Puede depender de	Evitar
Core	Nada o tipos muy basicos	Referencias a escenas, UI o reglas grandes.
Models	Core	UnityEngine, GameObject, UI, Input.
Data	Core y Models	Logica de combate pesada.
Views	Core, Models para leer estado, UnityEngine	Modificar reglas del juego.
Controllers	Core, Models, Data, Views, Services	Convertirse en la unica clase gigante.
Services	Core y APIs necesarias	Depender de una pantalla concreta.

Dependencia recomendada

Flujo	Lectura
Data -> Models	Los datos editables crean o alimentan modelos.
Controllers -> Models	Los controladores ejecutan acciones sobre el estado.
Controllers -> Views	Los controladores piden que la UI se actualice.
Views -> Models	Las vistas pueden leer estado para mostrarlo, pero no cambiar reglas.

3. Clases sugeridas para empezar

Estas clases son sugerencias de diseno, no codigo obligatorio. El equipo puede ajustar nombres segun avance el juego, pero conviene mantener el sufijo que indica el rol.

Prioridad	Clase sugerida	Carpeta	Responsabilidad
Alta	GameState	Core	Estados generales: menu, batalla, pausa, victoria, derrota.
Alta	BattlePhase	Core o Models	Fases internas de una batalla: turno jugador, turno boss, fin.
Alta	PokemonModel	Models	Vida, stats actuales y estado de un pokemon en runtime.
Alta	BattleModel	Models	Estado de la batalla: jugador, boss actual, turno y resultado.
Alta	PokemonDefinition	Data	Datos editables desde Unity para crear pokemon.
Alta	BossRushDefinition	Data	Lista ordenada de bosses y configuracion inicial.
Alta	BattleController	Controllers	Coordina inicio de batalla, acciones y cambios de turno.
Alta	BattleHudView	Views	Muestra barras de vida, textos de turno y mensajes.
Media	HealthBarView	Views	Componente reutilizable para vida actual/maxima.
Media	PokemonView	Views	Sprite, animaciones y feedback visual del pokemon.
Media	PlayerInputController	Controllers	Traduce botones/teclas en acciones del controlador.
Baja	SceneLoaderService	Services	Carga de escenas cuando el flujo lo necesite.
Baja	SaveService	Services	Guardado de progreso, cuando ya haya progreso real.

4. Acomodo recomendado dentro de cada carpeta

Al principio alcanza con las seis carpetas actuales. Cuando una carpeta tenga muchos archivos, recien ahi conviene dividirla en subcarpetas.

Carpeta	Subcarpetas futuras posibles	Cuando crearlas
Models	Battle, Pokemon, Inventory	Cuando haya varias reglas por sistema.
Views	UI, World, Effects	Cuando se mezclen HUD, sprites y efectos.
Controllers	Battle, Menu, Player	Cuando haya varios flujos de input o pantallas.
Data	Pokemon, Attacks, Bosses, Items	Cuando haya muchos ScriptableObject.
Services	Persistence, Audio, Scenes	Cuando los servicios crezcan o sean varios.

Regla practica: no crear subcarpetas vacias por adelantado. Primero dejar que aparezca una necesidad real; despues ordenar.

5. Reglas de programacion para mantener orden

Regla	Motivo
Una clase debe tener una responsabilidad clara.	Evita clases enormes dificiles de probar.
Los Models no deben usar UnityEngine salvo que sea inevitable.	Permite probar reglas sin escena.
Las Views no calculan resultados de combate.	Su trabajo es mostrar, no decidir.
Los Controllers coordinan, pero no guardan todo el estado.	El estado debe vivir en Models o Services.
Los datos configurables van a Data.	Evita hardcodear stats o valores de diseno.
Evitar nombres genericos como Manager si no hay alternativa.	Controller, Service, View o Model dicen mejor el rol.
Asignar referencias desde Inspector cuando sea posible.	Reduce busquedas globales y errores en runtime.
No agregar sistemas grandes sin un flujo minimo probado.	Permite avanzar con menos deuda tecnica.

6. Reglas para nombres

Tipo	Formato sugerido	Ejemplos
Modelo	Nombre + Model	PokemonModel, BattleModel, InventoryModel
Vista	Nombre + View	BattleHudView, HealthBarView, PokemonView
Controlador	Nombre + Controller	BattleController, MenuController
Datos	Nombre + Definition o Data	PokemonDefinition, AttackDefinition
Servicio	Nombre + Service	SaveService, SceneLoaderService
Estado/enum	Nombre claro sin sufijo forzado	GameState, BattlePhase, AttackType

7. Orden sugerido de implementacion

1. Crear datos minimos en **Data**: definicion de pokemon y definicion de boss rush.
2. Crear modelos minimos en **Models**: pokemon en runtime y batalla actual.
3. Crear una vista simple en **Views**: textos y barras de vida, sin animaciones todavia.
4. Crear un controlador en **Controllers**: iniciar batalla y ejecutar una accion simple.
5. Conectar todo en una escena de prueba, sin menus complejos.
6. Recien despues sumar ataques, items, animaciones, sonido, guardado o mas pantallas.

8. Que dejar para mas adelante

Sistema	Por que esperar
Inventario completo	Necesita reglas de items y UI adicional; puede trabar el primer avance.
Guardado de partida	Primero debe existir progreso real para guardar.
Multiples escenas	Una escena de prueba alcanza para validar el combate base.
Animaciones avanzadas	Mejor conectar cuando la regla de batalla ya funcione.
Balance complejo	Primero cerrar formulas simples; despues ajustar dificultad.
Arquitectura con inyeccion de dependencias	Usarla solo cuando haya suficiente complejidad para justificarla.

9. Checklist antes de crear una clase

Pregunta	Decision
Que responsabilidad exacta tiene?	Elegir Model, View, Controller, Data, Service o Core.
Necesita UnityEngine?	Si lo necesita, probablemente no es Model.
Debe verse en pantalla?	Crear o modificar una View.
Responde a input o botones?	Crear o modificar un Controller.
Es un valor que cambia el diseno del juego?	Moverlo a Data.
Va a ser usado por varias partes?	Evaluar Service o Core.

10. Checklist antes de subir cambios

Revision	Esperado
No hay reglas de juego dentro de Views.	OK
Los Models no buscan GameObjects ni leen input.	OK
El Controller no duplica datos del Model.	OK
Los datos editables no estan hardcodeados sin razon.	OK
Los nombres indican el rol de la clase.	OK
La escena de prueba funciona antes de sumar otra feature.	OK

11. Resumen para arrancar

Primero: datos minimos, modelo de batalla y una vista simple. **Despues:** controlador que conecte todo en una escena de prueba. **Ultimo:** sistemas secundarios como inventario, guardado, menus y animaciones. El objetivo inicial no es tener todo; es tener un flujo chico, claro y extensible.